

Final Project Report — Fileless Malware: Attack Simulation & Detection

Team: Hack the Blocks

Authors: Zac Davis, Caleb Clauson

Course: IT 359 — Spring 2026

Date: 2026-05-07

Introduction & Purpose (The What)

This report documents the design, implementation, and analysis of our course project exploring fileless malware techniques and behavior-based detection. The project has two complementary strands:

1. A red-team simulation that demonstrates common fileless techniques implemented in PowerShell, including in-memory code execution and periodic HTTP beaconing to a command-and-control (C2) server.
2. A blue-team detection suite implemented in Python that includes a deterministic, behavior-based detector and a companion machine-learning-assisted detector that leverages the Google Gemini API.

The primary goals are:

1. To build a compact, reproducible lab demonstrating how fileless techniques operate at a system and network level.
2. To evaluate practical detection strategies that focus on observable behaviors (process command lines, network endpoints, and beacon-like timing patterns) rather than static file signatures.
3. To provide an educational, auditable codebase and a test harness for further research into behavior-based and ML-driven detection.

All work produced for this project is strictly for controlled-lab educational use. See the `docs/setup_guide.md` and the disclaimer in the repository for safety guidance.

Technical Implementation (The How)

This section explains the structure and implementation details of each major component, with references to the source files in the repository.

Architecture Overview

The repository is organized into clear red-team and blue-team components. The red-team code simulates an in-memory payload and a simple C2 server. The blue-team code monitors host activity and attempts to surface suspicious behavior. The core files are:

1. [src/red_team/fileless_simulation.ps1](#) — PowerShell-based simulation of fileless techniques.
2. [src/red_team/c2_listener.py](#) — Flask-based C2 listener and simple operator interface.
3. [src/blue_team/detector.py](#) — Rule-based, behavior-focused detector implemented with psutil.
4. [src/blue_team/ml_detector.py](#) — ML-assisted detector that prepares features and queries an LLM (Gemini) for classification.

Each component is designed to be run independently in a contained lab environment (virtual machines, isolated networks). The PowerShell simulation defaults to DemoMode to avoid making system changes during routine testing.

Red Team: Simulation Script

The PowerShell script ([src/red_team/fileless_simulation.ps1](#)) intentionally demonstrates several concepts while remaining non-destructive by default:

1. Parameters: `-DemoMode` (default), `-LiveMode`, `-Undo`, `-C2Server`, and `-C2Port`. By default the script logs actions rather than executing destructive operations.
2. In-memory reconnaissance (`Get-SystemRecon`): The script collects system and environment information into in-memory data structures and returns them without writing to disk.
3. HTTP beaconing (`Send-Beacon` and `Start-BeaconLoop`): The simulated payload issues periodic HTTP POST requests to a configured C2 endpoint to register and send heartbeat information.
4. In-memory execution (`Invoke-MemoryExecution`): Illustrates how a script block can be constructed and executed directly in PowerShell process memory.
5. Environment variable payloads and registry-based persistence: Demonstrative functions show how environment variables or registry keys might be used for persistence (these are not executed in DemoMode).

The script is intentionally explicit about its educational nature and defaults to safe behavior. See [src/red_team/fileless_simulation.ps1](#) for function-level comments and the demo-to-live flags.

Red Team: C2 Listener

The C2 server at [src/red_team/c2_listener.py](#) is a minimal Flask application exposing a few endpoints:

1. POST /register: Accepts reconnaissance JSON from an agent and records the session.
2. POST /heartbeat: Receives periodic status updates and optionally returns queued commands.
3. GET /sessions: Returns the in-memory session table for inspection.
4. GET/POST /commands/<session_id>: Allows queuing commands for a given session.

Server state is kept in simple in-memory structures for clarity: SESSIONS, HEARTBEATS, and COMMAND_QUEUE. The listener is suited for lab testing but not for long-term or production use; it purposefully stores data in process memory for the demo.

Implementation notes:

1. The Flask app uses jsonify and logs events via Python's logging module with a --verbose flag to increase verbosity.
2. The app.run(..., use_reloader=False) call ensures only a single process is started when run as a CLI program.

Blue Team: Rule-Based Detector (detector.py)

The behavior-based detector is intentionally concise and auditable. Key implementation details:

1. Process enumeration uses psutil.process_iter(attrs=["pid", "name"]) with safe exception handling during iteration.
2. PowerShell process identification is handled by is_powershell_process(proc), which checks the lowercase process name against a small set: powershell.exe, powershell, pwsh.exe, and pwsh.
3. Suspicious command-line detection uses a small but focused set of regular expressions (SUSPICIOUS_POWERSHELL_PATTERNS) which match common flags and patterns used in fileless payloads, for example -nop, -nopprofile, -w hidden, -enc(oddedcommand)?, and invoke-expression|iex.
4. C2 detection inspects proc.net_connections(kind="inet") and compares remote addresses/ports against a configured C2 host/port. If a process has a network remote address that matches the configured C2 port (and host), it is flagged.
5. Dedupe logic: a seen dictionary maps pid -> cmdline so the detector doesn't repeatedly print the same alert for an unchanged process.
6. Output: alerts are formatted with a clear banner and include pid, process_name, severity, reason, and the truncated cmdline.

The choice to focus on command-line patterns and remote endpoints leads to a small, explainable rule set that is easy to validate. The code is in [src/blue_team/detector.py](#).

Blue Team: ML-Assisted Detector (`ml_detector.py`)

The ML detector is structured as a layered system that combines cheap, local heuristics with selective AI calls. The high-level flow is:

1. A fast local scoring function (`local_risk_score`) assigns inexpensive signals (powershell-family process, suspicious flags, remote endpoints, beacon-like timing) into a numerical score and short rationale.
2. If the local score exceeds a configurable threshold (`ML_DETECTOR_MIN_SCORE`) the process is considered a candidate for AI-assisted analysis. This throttles expensive API calls.
3. For selected candidates, the detector builds a structured prompt (`build_model_prompt`) containing anonymized process features, recent connection timestamps, and local reasoning, and sends that prompt to the Google Gemini API (via the `google-genai` client) for classification.
4. The model is expected to return a compact JSON-like summary which the code parses, extracts a verdict and confidence, and formats an alert for the operator.

Implementation notes and safeguards:

1. The detector logs machine-readable JSONL records via a `RotatingFileHandler` for post-hoc analysis and ingestion into SIEM tools.
2. Several environment variables drive runtime tuning: `ML_DETECTOR_POLL_SECONDS`, `ML_DETECTOR_MIN_SCORE`, `ML_DETECTOR_MAX_AI_PER_CYCLE`, and logging-related variables.
3. The AI stage is intentionally designed for *explainability*: the prompt asks for a short list of reasons and a numeric score, so the operator can reconcile the model output with local heuristics.

The ML detector code lives in [src/blue_team/ml_detector.py](#). It is a prototype for integrating large language models into a detection pipeline and is not intended as a finished product.

Justification & Analysis (The Why)

This section explains the design choices, trade-offs, and expected effectiveness of the approaches used.

Why behavior-based detection?

Traditional signature-based tools match static files or known byte sequences. Fileless malware purposely avoids writing persistent executable files, so signatures applied to disk-resident artifacts are often ineffective. Behavior-based signals (command-line flags, process ancestry, network endpoints, and temporal beaconing patterns) provide features that persist even when the attacker never writes an executable to disk.

Key advantages:

1. Observable at run-time: command-line arguments, network sockets, and process activity are available to monitoring agents.
2. Harder to obfuscate at scale: while attackers can obfuscate individual command-lines, they still must perform operations that manifest in process actions and network traffic.

Limitations:

1. Higher false-positive risk: legitimate administration scripts and automation tools can use similar flags or network endpoints.
2. Evasion: sophisticated adversaries can mimic normal traffic timing, use encrypted channels, or stage activity through other trusted processes.

We mitigate these limitations by combining multiple orthogonal signals (command-line heuristics + C2 network detection + beacon-likeness temporal analysis) and by providing an ML-assisted escalation path to provide richer context to operators when local heuristics are ambiguous.

Trade-offs and thresholds

1. Local-score thresholds (ML_DETECTOR_MIN_SCORE) balance the number of AI queries and the likelihood of catching subtle behavior. A lower threshold increases recall but consumes more API calls and increases operator overhead from model outputs.
2. `max_ai_per_cycle` prevents the system from exhausting API quota in noisy environments.
3. Deduplication (seen mapping) reduces alert fatigue by avoiding repeated alerts for unchanged process state.

False positives and evaluation

A proper evaluation requires labeled examples: benign administrative PowerShell activity, common IT automation, and a set of simulated malicious beacons. For this project we recommend:

1. Building a small corpus of benign PowerShell usage from test VMs.
2. Running the simulation in DemoMode against the C2 listener to produce controlled malicious traces.
3. Measuring precision/recall at different scoring thresholds and tracking the AI model's impact on operator decisions.

Differences From Real-World Attacks

We designed the simulation and detectors for clarity and safety, and they intentionally differ from a full adversary lifecycle in several important ways. It is important to be explicit about these differences so results are interpreted correctly.

1. **Delivery and access:** In the lab, we explicitly invoke the PowerShell simulation and generally rely on manual steps to place the script on the test host. In contrast, a real attacker typically uses social engineering, exploit chains, or supply-chain techniques to gain initial execution without explicit operator consent. Our simulation omits the nuances of initial access for safety and reproducibility.
2. **Operational security and stealth:** A real threat actor will invest heavily in stealth (timing jitter, encrypted custom protocols, living-off-the-land binaries, signed components, or covert channels) and in adapting behavior based on defender telemetry. Our simulated beacon uses plain HTTP for clarity and to make network detection tractable in a lab; it is not representative of the broad set of C2 channels used in sophisticated intrusions.
3. **Defensive constraints:** In a controlled lab we sometimes need to relax protections (for example, running with reduced endpoint protection, or adjusting environment variables) to enable observation and debugging. Real defenders cannot simply disable endpoint protection at scale; they must design detection and response that operate alongside active protection.
4. **Scale and persistence:** Enterprise-scale attacks often involve lateral movement and multi-host persistence. This project focuses on single-host behavior and simple C2 interactions for tractability.

By documenting these differences we avoid overstating the results: the project demonstrates detection concepts but does not claim to replicate an advanced persistent threat at scale.

Conclusion

This project provides a compact, auditable lab illustrating fileless malware concepts and behavior-based defenses. Our principal contributions are:

1. A readable red-team simulation that demonstrates core fileless techniques while defaulting to safe DemoMode.
2. A deterministic detector that targets clear, explainable signals with a low operational footprint.
3. A prototype ML-assisted escalation pipeline that combines local heuristics with an LLM-backed contextual classifier, along with practical throttles and logging.

Suggested next steps and improvements:

1. Integrate the detectors with endpoint telemetry sources (Sysmon/ETW) for richer signals.

2. Develop a labeled dataset for robust evaluation and retraining of the ML escalation model.
3. Add network-level detection rules for encrypted channels (TLS fingerprinting, JA3/JA3S) and DNS-based C2 discovery.
4. Extend the red-team simulation to provide multiple delivery vectors and more realistic C2 transport options while keeping safe execution modes.

We believe this codebase is a practical foundation for continued experimentation in behavior-based detection and for grounding research questions about how best to combine lightweight local heuristics with selective ML assistance.

Appendix: Reproducing the Lab (Safe, Local Steps)

The following commands are intended for isolated lab VMs only.

1. Create and activate a Python virtual environment:

```
python -m venv venv
# macOS/Linux
source venv/bin/activate
# Windows PowerShell
.\venv\Scripts\Activate.ps1
```

1. Install Python dependencies:

```
pip install -r requirements.txt
```

1. Start the C2 listener (on the attacker machine):

```
python src/red_team/c2_listener.py --host 0.0.0.0 --port 8080
```

1. Run the detector on the target VM (adjust --c2-host as needed):

```
python src/blue_team/detector.py --c2-host 127.0.0.1 --c2-port 8080 --debug
```

1. Run the PowerShell simulation on a Windows test VM in DemoMode:

```
# From PowerShell
.\src\red_team\fileless_simulation.ps1 -DemoMode -C2Server 127.0.0.1 -C2Port 8080
```

All components include help text and flags; consult the top of each source file for usage notes.

References?